



The *-Files: Call for Anonymizing Prompts

Build an LLM-based system that finds and masks sensitive data in a document, preserving its meaning and formatting.

EVENT	JunctionX Lisbon 2026
WHEN	10-11 October 2026
WHERE	IST — Campus Taguspark, Porto Salvo, Lisbon

The challenge

Challenge one-liner: build an LLM-based system that identifies and masks sensitive or personal data in a document, keeping the document's meaning and formatting intact.

INPUT

1 shared document

OUTPUT

Masked text + prompt

PITCH

2 minutes max.

Introduction

Every team receives the same text document, filled with fictitious personal data: [raw_data/document_to_anonymize.pdf](#). Your system must replace the sensitive data with asterisks and return the document with its original formatting. The output is scored automatically when you open a pull request against [main](#).

Questions can be asked to the NOS mentors on site, or on the organization's Discord.

The core of the challenge

1. **Detect.** Use prompt engineering to make the LLM recognize patterns and reason about the context around sensitive data, beyond its format.
2. **Mask.** Replace each anonymized word with a single `*`. A two-word name becomes two asterisks:

```
Ana Correia → * *
```

3. **Preserve.** Keep the original formatting and the document's meaning. The only change in layout is that empty lines are removed. A correct output starts like this:

```
Relatório de Admissão - Centro Médico Lisboa
Data: 15 de abril de 2025
Referência: ADM-2025-04-15-089
Informações do Paciente:
```

4. **Validate (optional).** Add a manual validation step to handle false positives and false negatives.

Suggested technical components

- **Model:** Google Gemini, through the Gemini API (tutorial in [tutorials/gemini_tutorial.md](#)).
- **PDF to text:** PyMuPDF, as shown in [tutorials/pdf_to_txt.ipynb](#).
- **Environment:** Python script or Jupyter / Colab notebook.
- **Version control:** Git and GitHub (tutorial in [tutorials/git_tutorial.md](#)).
- Open-source frameworks and packages are allowed, as long as they are referenced.

Guidelines

Before you start

You need	How
GitHub account	github.com/signup
Google account	accounts.google.com/signup
Google API key	aistudio.google.com → Get API Key → Create API key (under a new project)

Save the generated API key in a safe place — you will need it later.

Repository layout

Path	What it holds
<code>raw_data/</code>	The document to anonymize.
<code>submission/</code>	<code>submission.txt</code> , <code>prompt.txt</code> and your code.
<code>tutorials/</code>	Problem statement, Gemini tutorial, Git tutorial, PDF-to-text notebook.
<code>.github/workflows/</code>	The automatic evaluation workflows.

Getting the repository

Repository: github.com/nosportugal/nos-gen-ai-hackathon

1. **Fork** the repository to your GitHub account. Each team has one fork: one member owns it and adds the others as collaborators.
2. **Clone** your fork and create a working branch, e.g. `team-<name>`.
3. Never work directly on `main`.
4. Work inside `submission/`.

Your fork is your team's workspace

We will assess collaboration and code quality from the commit history, branches and merges of your fork. Every member should commit from their own account. Keep the fork public and preserve its history: avoid squashing or force-pushing over it.

Submission

What to deliver

Edit the two provided files and add your code:

File	Content
<code>submission/submission.txt</code>	Only the output generated by your model or API, as your system returns it. This is what the judges evaluate.
<code>submission/prompt.txt</code>	The exact prompt used to generate that output.
<code>submission/your-code</code>	The notebook or script (<code>.py</code> , <code>.ipynb</code>) used to develop the solution. It is part of the evaluation.

Do not rename, move or delete the provided files.

Keep submission.txt clean

Do not include the team name, project title, GitHub links, team members, any explanation or any extra formatting.

How to submit

1. Run `git branch` and confirm you are on your own working branch.
2. Check the file status: `git status`.
3. Stage the files: `git add submission/submission.txt submission/prompt.txt submission/<your-code>`.
4. Commit: `git commit -m "Final submission -Team Name-"`.
5. Push: `git push origin <your-branch>`.
6. Open a **pull request** from your branch to `main` of the **original** repository (`nosportugal/nos-gen-ai-hackathon`).
7. Register your submission on the JunctionX platform, using the link of your pull request as the repository link.

Opening the pull request triggers the automatic evaluation workflows, so you get a score as soon as you submit. The final submission must be in before the end of the challenge.

Evaluation and rules

Evaluation criteria

Criterion	Weight
Technical implementation	40%
Precision — detection hit rate	10%
Prompt — prompt and response quality	10%
Anonymization quality — context preserved after removal	5%
Explicability — clarity and technical rigor of the solution	5%
Efficiency — optimized use of prompts and engineering techniques	5%
GitHub — code quality: structure, branches, merges, commits	5%
Collaboration — shared commit history and integration between members	30%
Documentation — clarity, organization and technical detail (README, usage, setup)	10%
Pitch creativity — originality, impact and communication	20%

The Collaboration and GitHub rows are judged from your fork's history. The other rows are judged from your pull request.

Automatic evaluation

Three GitHub Actions workflows run on your pull request:

- **Hackathon Evaluation** — compares `submission/submission.txt` with a hidden reference.
- **Prompt Evaluation** — scores `submission/prompt.txt`.
- **Lint Python Code** — runs `flake8` on the repository.

A green check can hide a missing file

If `submission.txt` or `prompt.txt` is missing, the matching workflow is skipped and still reports success, so a green check does not guarantee a score. Make sure both files are in your pull request.

General rules

- Open-source frameworks and packages are allowed, as long as they are referenced.
- Every project must include clear documentation: README, usage and setup instructions.
- The commit history is analyzed to evaluate good collaborative practices.
- The final pitch lasts at most 2 minutes per team.
- Failure to comply with the rules may mean immediate disqualification.

Open-source intellectual property

- **Open-source license:** the project's source code must be released under an open-source license.
- **Public repository:** the code must be in a public repository (e.g. GitHub).
- **Repurpose:** the parties may reuse the work for other non-commercial purposes.

Prizes

The winning team receives tickets for NOS Alive 2027, one for each team member.